ZESPÓŁ SZKÓŁ ELEKTRYCZNO-MECHANICZNYCH

Pracownia testowania i dokumentowania aplikacji

Testowanie aplikacji

Autor: Marcin Migacz

Klasa: 5P

Prowadzący: mgr inż. Kamil Wojnarowski

NOWY SĄCZ 2026r.

Spis treści

Wstęp 4

1. Testy jednostkowe 5

1.1. Implementacja programu 5

1.2. Implementacja testów 9

1.3. Git 11

Wstęp

Testowanie oprogramowania to proces sprawdzania, czy program działa zgodnie z wymaganiami, oraz wykrywania w nim błędów przed przekazaniem go użytkownikom. Błąd (defekt) to wada w kodzie, która może doprowadzić do awarii, czyli widocznego niepoprawnego działania systemu. Celem testowania jest nie tylko znajdowanie usterek, ale też ograniczanie ryzyka jego wdrożenia. Ze względu na poziom wyróżnia się testy jednostkowe, integracyjne, systemowe i akceptacyjne, które sprawdzają kolejno pojedyncze elementy, ich współpracę, cały system oraz zgodność z oczekiwaniami klienta. Ze względu na cel dzielimy je na funkcjonalne (co system robi) i niefunkcjonalne (jak działa: wydajność, bezpieczeństwo, użyteczność), a osobną grupę stanowią testy regresyjne, które po zmianach w kodzie sprawdzają, czy nie zepsuto istniejących funkcji. Wyróżnia się też podejścia czarnej skrzynki (bez znajomości kodu), białej skrzynki (z analizą wnętrza programu) i szarej skrzynki (mieszane). Pierwsza zasada testowania mówi, że testy ujawniają obecność błędów, ale nie dowodzą ich braku. Druga głosi, że testowanie wyczerpujące jest niemożliwe, więc wysiłek kieruje się na obszary o największym ryzyku. Trzecia zasada to wczesne testowanie, bo błąd znaleziony na etapie projektu kosztuje znacznie mniej niż ten znaleziony po wdrożeniu. Kolejne zasady mówią, że defekty grupują się w niewielu modułach, a wielokrotne uruchamianie tych samych testów przestaje wykrywać nowe błędy. Ostatnie zasady to zależność testów od kontekstu oraz złudzenie braku błędów: program bez usterek nadal nie jest wartościowy, jeśli nie spełnia potrzeb użytkownika. Proces testowy zaczyna się od planowania i nadzoru, czyli określenia zakresu, zasobów i kryteriów zakończenia. Następnie następuje analiza i projektowanie testów, w których powstają przypadki testowe z danymi wejściowymi i oczekiwanymi wynikami. Potem testy są implementowane i wykonywane, a wyniki rzeczywiste porównuje się z oczekiwanymi, po czym raportuje się znalezione błędy, wykonuje retesty po poprawkach i zamyka proces podsumowaniem.

1. Testy jednostkowe

Czym są? - Testy jednostkowe (ang. *unit tests*) sprawdzają najmniejsze samodzielne fragmenty programu, np. pojedynczą metodę lub klasę, w izolacji od reszty systemu. Każdy test podaje określone dane wejściowe, uruchamia testowaną jednostkę i porównuje jej wynik z oczekiwanym. Są szybkie, w pełni automatyczne i powtarzalne, więc można je uruchamiać po każdej zmianie kodu.

Kiedy je wykonujemy? - Najczęściej w trakcie pisania kodu, zwykle przez samego programistę, a potem po każdej modyfikacji oraz automatycznie w procesie CI/CD przed scaleniem zmian. Dzięki temu błąd jest wykrywany w miejscu, w którym powstał i jest tani w naprawie.

Narzędzie - xUnit. xUnit to popularny, darmowy framework do testów jednostkowych w .NET (C#). Testy są zwykłymi metodami oznaczonymi atrybutem [Fact] (test bez parametrów) lub [Theory] (test z wieloma zestawami danych). Do sprawdzania wyników służą asercje klasy Assert. Test przechodzi, gdy wszystkie asercje są spełnione, a nie przechodzi, gdy któraś zawiedzie lub wystąpi nieoczekiwany wyjątek.

1.1. Implementacja programu

Program to konsolowa loteria liczbowa typu „Lotto". Użytkownik podaje, ile zestawów chce wylosować. Program losuje dla każdego zestawu 6 różnych liczb z przedziału 1–49 i zapisuje je w dwuwymiarowej tablicy (wiersz = zestaw, kolumna = pozycja w zestawie). Następnie wyświetla wszystkie zestawy i zlicza, ile razy w całym losowaniu wystąpiła każda liczba od 1 do 49. Gdy użytkownik wpisze tekst zamiast liczby, program wypisuje komunikat o błędzie i kończy działanie.

Przykładowy wynik dla wejścia 2:

```
Ile wygenerować losowań?
2
Zestawy wylosowanych liczb:
Losowanie 1: 7 23 4 41 19 35
Losowanie 2: 12 3 48 27 7 15
Wystąpienia liczby 1: 0
Wystąpienia liczby 2: 0
Wystąpienia liczby 3: 1
(Wypisywane wystąpienia kolejnych liczb aż do 49)
```

Opis kodu:

- **Stałe:**

LiczbWZestawie	6	Ile liczb zawiera jeden zestaw (jedno losowanie).
MinLiczba	1	Najmniejsza liczba, którą można wylosować.
MaxLiczba	49	Największa liczba, którą można wylosować

- Funkcja Main

1. int.TryParse(Console.ReadLine(), out int ileZestawow) - wczytuje tekst z klawiatury i próbuje zamienić go na liczbę całkowitą. Jeśli się uda, zwraca true, a wynik trafia do zmiennej ileZestawow. Jeśli użytkownik wpisze tekst (np. abc), zwraca false i program nie ulega awarii.
2. **if (poprawne dane):**
    - new int[ileZestawow, LiczbWZestawie] tworzy dwuwymiarową tablicę losowania o n wierszach (zestawy) i 6 kolumnach (liczby w zestawie).
    - Pętla zewnętrzna for (i...) wykonuje się raz dla każdego zestawu. W każdym obiegu wywołuje LosujZestaw(), która zwraca 6-elementową tablicę.
    - Pętla wewnętrzna for (j...) przepisuje 6 wylosowanych liczb do wiersza i tablicy losowania.
    - WyswietlLosowania(ileZestawow, losowania) wypisuje wyniki i statystykę.
3. **else (błędne dane):** wypisuje komunikat o niepoprawnej wartości i kończy program instrukcją return.

• **Funkcja LosujZestaw()** - losuje jeden zestaw 6 różnych liczb z przedziału 1–49. Nie przyjmuje argumentów i zwraca tablicę int[] z wylosowanymi liczbami.

1. **int[] zestaw = new int[LiczbWZestawie]** - tworzy pustą tablicę na 6 liczb. Początkowo wszystkie elementy mają wartość 0.
2. **Random generator = new Random()** - tworzy generator liczb pseudolosowych.
3. **int i = 0** to licznik zapisanych już liczb, a zarazem indeks następnego wolnego miejsca w zestawie.
4. Pętla **while (i < LiczbWZestawie)** działa, dopóki zestaw nie zawiera 6 liczb.
5. **generator.Next(MinLiczba, MaxLiczba + 1)** losuje liczbę całkowitą. Górna granica w Next wynosi o 1 mniej niż dana liczba, dlatego dodano + 1, żeby 49 mogło zostać wylosowane.
6. **zestaw.Contains(wylosowana)** sprawdza, czy taka liczba już jest w zestawie:

    jeśli tak, **continue** przerywa obieg pętli i losowanie zaczyna się od nowa, a licznik i się nie zwiększa,

    jeśli nie, liczba zostaje zapisana pod indeksem i, a następnie i++ przesuwa licznik.

7. **return** zestaw zwraca gotowy zestaw.

• **Funkcja WyswietlLosowania()** – losuje

1. **int[] wystapienia = new int[MaxLiczba]** - tworzy tablicę 49 liczników. Indeks liczba - 1 odpowiada liczbie z loterii, czyli liczba 1 jest pod indeksem 0, a liczba 49 pod indeksem 48.
2. Pierwsza pętla for zeruje wszystkie liczniki.
3. Zagnieżdżone pętle for przechodzą po wszystkich zestawach (i) i liczbach w zestawie (j):
    - **Console.Write($"Losowanie {i + 1}: ")** - wypisuje numer losowania, numerowany od 1 dla użytkownika,
    - **wystapienia[losowania[i, j] - 1]++** - zwiększa licznik wylosowanej liczby o 1,
    - **Console.Write(losowania[i, j] + " ")** - wypisuje liczbę, a po niej spację,
    - **Console.WriteLine("")** - - po 6 liczbach przechodzi do nowej linii.
4. Ostatnia pętla for wypisuje dla każdej liczby od 1 do 49 tekst „Wystąpienia liczby X: Y”, gdzie Y to wartość licznika.

```
using System;

namespace Loteria
{
    Odwołania: 4
    public class Program
    {
        Odwołania: 5
        const int LiczbWZestawie = 6;
        1 odwołanie
        const int MinLiczba = 1;
        Odwołania: 2
        const int MaxLiczba = 49;

        Odwołania: 0
        public static void Main(string[] args)
        {
            Console.WriteLine("Ile wygenerować losowań?");

            if (int.TryParse(Console.ReadLine(), out int ileZestawow))
            {
                int[,] losowania = new int[ileZestawow, LiczbWZestawie];

                for (int i = 0; i < ileZestawow; i++)
                {
                    int[] zestaw = LosujZestaw();
                    for (int j = 0; j < LiczbWZestawie; j++)
                    {
                        losowania[i, j] = zestaw[j];
                    }
                }

                WyswietlLosowania(ileZestawow, losowania);
            }
            else
            {
                Console.WriteLine("Niepoprawna wartość. Podaj liczbę całkowitą z zakresu 1-49");
                return;
            }
        }

        Odwołania: 4 | 3/3 passing
        public static int[] LosujZestaw()
        {
            int[] zestaw = new int[LiczbWZestawie];
            Random generator = new Random();
            int i = 0;

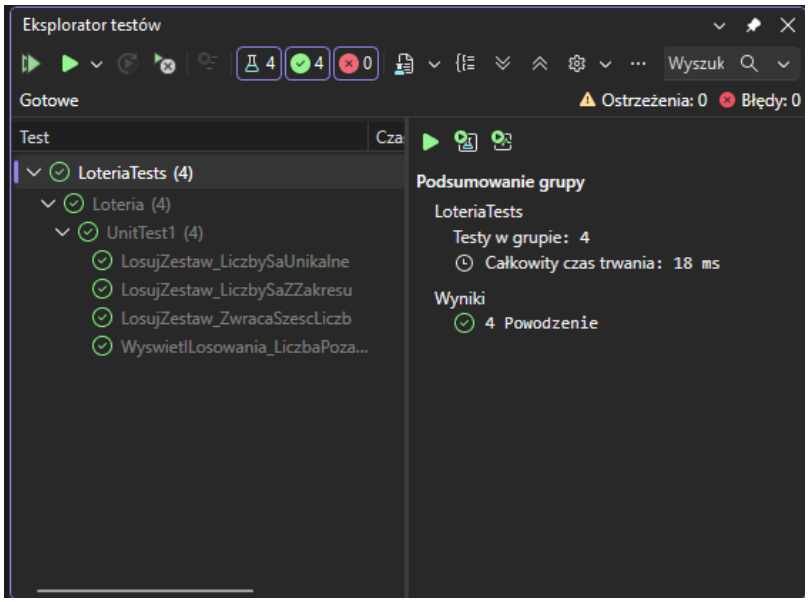
            while (i < LiczbWZestawie)
            {
                int wylosowana = generator.Next(MinLiczba, MaxLiczba + 1);
                if (zestaw.Contains(wylosowana))
                {
                    continue;
                }
                else
                {
                    zestaw[i] = wylosowana;
                }

                i++;
            }
            return zestaw;
        }
```

```
59
                Odwołania: 2 | 1/1 passing
60              public static void WyswietlLosowania(int liczbaZestawow, int[,] losowania)
61              {
62                  Console.WriteLine("Zestawy wylosowanych liczb:");
63                  //[indeks + 1 -> liczba, jej wystąpienia]
64                  int[] wystapienia = new int[MaxLiczba];
65
66                  for(int i = 0; i < wystapienia.Length; i++)
67                  {
68                      wystapienia[i] = 0;
69                  }
70
71                  for (int i = 0; i < liczbaZestawow; i++)
72                  {
73                      Console.Write($"Losowanie {i + 1}: ");
74
75                      for (int j = 0; j < LiczbWZestawie; j++)
76                      {
77                          //Zwiększenie liczby wystąpień danej liczby
78                          wystapienia[losowania[i, j] - 1]++;
79
80                          Console.Write(losowania[i, j] + " ");
81                      }
82
83                      Console.WriteLine("");
84                  }
85
86                  for (int i = 0; i < wystapienia.Length; i++)
87                  {
88                      Console.WriteLine($"Wystąpienia liczby {i + 1}: {wystapienia[i]}");
89                  }
90
91              }
92          }
93      }
94
```

1.2. Implementacja testów

#	Typ	Metoda testowa	Opis	Oczekiwany wynik
1	poprawny	LosujZestaw_ZwracaSzescLiczb	Wywołuje LosujZestaw() i sprawdza Assert.Equal(6, zestaw.Length).	Zestaw ma dokładnie 6 elementów. Test zdany.
2	poprawny	LosujZestaw_LiczbySaUnikalne	Losuje 1000 zestawów. Distinct() usuwa powtórzenia, więc liczba elementów po jego zastosowaniu musi być równa długości zestawu.	W żadnym zestawie nie ma duplikatów. Test zdany.
3	poprawny	LosujZestaw_LiczbySaZZakresu	Losuje 1000 zestawów i dla każdej liczby sprawdza Assert.True(liczba >= 1 && liczba <= 49).	Każda liczba mieści się w przedziale 1–49. Test zdany.
4	niepoprawny	WyswietlLosowania_LiczbaPoza_Zakresem_RzucaWyjatek	Dane zawierają liczbę 50, spoza zakresu. Metoda próbuje zwiększyć wystapienia[49], ale tablica ma indeksy 0–48. Assert.Throws sprawdza, że rzucany jest IndexOutOfRangeException.	Wyjątek zostaje rzucony, więc test jest zdany.



LoteriaTests — Loteria.UnitTest1

```
1   namespace Loteria
2   {
        Odwołania: 0
3       public class UnitTest1
4       {
5           // TEST 1 (poprawny): zestaw zawiera dokładnie 6 liczb
6           [Fact]
            Odwołania: 0
7           public void LosujZestaw_ZwracaSzescLiczb()
8           {
9               int[] zestaw = Program.LosujZestaw();
10
11              Assert.Equal(6, zestaw.Length);
12          }
13
14          // TEST 2 (poprawny): liczby w zestawie są unikalne
15          [Fact]
            Odwołania: 0
16          public void LosujZestaw_LiczbySaUnikalne()
17          {
18              //Przypadek powtarzany wiele razy, by ominąć element losowości
19              for (int powtorzenie = 0; powtorzenie < 1000; powtorzenie++)
20              {
21                  int[] zestaw = Program.LosujZestaw();
22
23                  Assert.Equal(zestaw.Length, zestaw.Distinct().Count());
24              }
25          }
26
27          // TEST 3 (poprawny): liczby w zestawie są z zakresu 1-49
28          [Fact]
            Odwołania: 0
29          public void LosujZestaw_LiczbySaZZakresu()
30          {
31              for (int powtorzenie = 0; powtorzenie < 1000; powtorzenie++)
32              {
33                  int[] zestaw = Program.LosujZestaw();
34
35                  foreach (int liczba in zestaw)
36                  {
37                      Assert.True(liczba >= 1 && liczba <= 49);
38                  }
39              }
40          }
41
42          // TEST 4 (niepoprawny): liczba 50 jest poza zakresem
43          [Fact]
            Odwołania: 0
44          public void WyswietlLosowania_LiczbaPoza_Zakresem_RzucaWyjatek()
45          {
46              int[,] losowania = { { 1, 2, 3, 4, 5, 50 } };
47
48              Assert.Throws<IndexOutOfRangeException>(() => Program.WyswietlLosowania(1, losowania));
49          }
50
51      }
52  }
53
```

1.3. Git

Screen z git log commitów wprowadzających przypadki testowe:

```
Admin@Marcin MINGW64 /c/Projekty/GIT/Marcin_Migacz_5P_gr2 (master)
$ git log
commit a254e49f4cf00c380d2539b707802bb305a0dcea (HEAD -> master, origin/master, origin/HEAD)
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Sep 30 23:42:15 2026 +0200

    Utworzenie czwartego testu - sprawdzenia czy wyrzuci błąd w przypadku liczby z poza zakresu

commit 023f5357a8ccbfb73ea8c129176f36e548eb348d
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Sep 30 23:40:20 2026 +0200

    Utworzenie trzeciego testu - sprawdzania czy liczby znajdują się w zakresie

commit 90a45633ff60601256086cb1bf2db6ab6413b76f
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Sep 30 23:28:06 2026 +0200

    Utworzenie drugiego testu - sprawdzania unikalności liczb w zestawie

commit 9ab7d1aecb41699b0fc3d3bf2d29d13ad7ad00d5
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Sep 30 23:24:09 2026 +0200

    Utworzenie pierszego testu - sprawdzania poprawnej długości zestawu liczb

commit 0f9bdeaa64052f8170365981a2ed3567d4d7871e
Author: MaRR1347 <mmigacz1347@gmail.com>
Date:   Wed Sep 30 23:14:55 2026 +0200

    Dodanie nietrywialnego programu - generatora losowań
```

2. Testy integracyjne

Czym są? – Testy integracyjne (ang. *integration tests*) sprawdzają, czy połączone moduły i komponenty aplikacji współpracują ze sobą prawidłowo. W przeciwieństwie do testów jednostkowych nie izolują kodu, lecz weryfikują jego interakcje z prawdziwymi elementami zewnętrznymi, takimi jak baza danych, pliki systemowe, zewnętrzne interfejsy API czy kolejki komunikatów.

Kiedy je wykonujemy? – Wykonuje się je po przejściu testów jednostkowych, gdy poszczególne moduły są już gotowe i połączone ze sobą. Są również uruchamiane automatycznie w potokach CI/CD (np. na osobnym środowisku testowym), aby upewnić się, że zmiana w jednym module nie zepsuła komunikacji z resztą systemu.

2.1. Implementacja programu

Ten rozdział będzie się opierać na programie opisanym w poprzednim punkcie. Szczegóły jego implementacji znajdują się w rozdziale 1.1. dokumentacji.

2.2. Implementacja testów